

ELF ARM - crackme 1337

Thursday, August 06, 2026 12:07 AM

On sait que r0 est un argument pour les fonctions et c'est aussi la valeur de retour.

Pour rappel :

shl \$0x4, %eax le décale de 4 bits vers la gauche, ce qui équivaut à le multiplier par 16 (2⁴)

Ne pas oublier que un INC eax, 1 en x86/x64 est comme ceci en ARM :

```
ldr r0, [something]
```

```
adds r0, 1
```

```
str r0, [something]
```

ldr = load

str = store

to run ARM locally

```
qemu-arm -L /usr/arm-linux-gnueabi -g 1234 ./ch23.bin AAAAAA
```

```
gdb-multiarch ch23.bin
```

```
(gdb) target remote :1234
```

une fois dans gdb il faut faire :

```
(gdb) file ch23.bin
```

```
; NULL XREF from entry0 @ +0x30(r)
```

```
r 456: int main (int argc, char **argv);
```

```
| ^ args(r0, r1) vars(10:sp[0x4..0x34])
```

```
| | 0x00008290 0dc0a0e1 mov ip, sp
```

```
| | 0x00008294 10d82de9 push {r4, fp, ip, lr, pc}
```

```
| | 0x00008298 04b04ce2 sub fp, ip, 4
```

```
| | 0x0000829c 24d04de2 sub sp, sp, 0x24
```

```
| | 0x000082a0 28000be5 str r0, [var_28h] ; 0x28 ; 40 ; argc
```

```
| | 0x000082a4 2c100be5 str r1, [var_2ch] ; 0x2c ; 44 ; argv
```

```
| | 0x000082a8 28301be5 ldr r3, [var_28h] ; 0x28 ; 40
```

```
| | 0x000082ac 010053e3 cmp r3, 1 ; 1
```

```
| |< 0x000082b0 020000ca bgt 0x82c0 => si on met un parametre en dessous de 1 on saute vers une erreur
```

```
| | 0x000082b4 0030e0e3 mvn r3, 0 => on fait un move NOT donc on bouge l'inverse du résultat dans r3
```

```
| | 0x000082b8 30300be5 str r3, [var_30h] ; 0x30 ; 48
```

```
| |< 0x000082bc 610000ea b 0x8448 => vers bad guy
```

```
| | | ; CODE XREF from main @ 0x82b0(x)
```

```
| |> 0x000082c0 0030a0e3 mov r3, 0 => on met 0 dans r3
```

```
| | 0x000082c4 1c300be5 str r3, [var_1ch] ; 0x1c ; 28 => on store dans var1ch la valeur contenue dans r3 donc 0 donc var1ch contient 0
```

```
| | 0x000082c8 2000a0e3 mov r0, 0x20 ; size_t size => on alloue 0x20 de taille
```

```
| | 0x000082cc dfffffeb bl sym.xmalloc ; void *xmalloc(size_t size) => on call une fonctionne pour allouer de la place.
```

```
| | 0x000082d0 0030a0e1 mov r3, r0 => c'est l'adresse de la mémoire allouée (un pointeur)
```

```
| | 0x000082d4 20300be5 str r3, [var_20h] ; 0x20 ; 32 => on pointe vers ces 32 octets alloués. donc var20h est un pointeur
```

```
| |< 0x000082d8 130000ea b 0x832c => on saute vers cette adresse
```

```
| | | ; CODE XREF from main @ 0x8334(x)
```

```
| |> 0x000082dc 1c301be5 ldr r3, [var_1ch] ; 0x1c ; 28 => on load la valeur contenue dans var1CH donc 0 donc r3 = 0
```

```
| | | 0x000082e0 0321a0e1 lsl r2, r3, 2 => on décale r3 de 2 bit vers la gauche et on met le résultat dans r2. donc on décale de 4 bit vers la gauche
```

```
| | | 0x000082e4 20301be5 ldr r3, [var_20h] ; 0x20 ; 32 => on récupère la valeur dans var20 et on la met dans r3. Vu que var20h est un pointeur on récupère l'adresse de la mémoire allouée je pense.
```

```
| | | 0x000082e8 024083e0 add r4, r3, r2 => on additionne l'adresse avec ce que contient r2 donc on se déplace dans la mémoire. et on met l'adresse dans r4
```

```
| | | 0x000082ec 2000a0e3 mov r0, 0x20 ; size_t size => on call une fonctionne pour allouer de la place.
```

```
| | | 0x000082f0 d4ffffeb bl sym.xmalloc ; void *xmalloc(size_t size)
```

```
| | | 0x000082f4 0030a0e1 mov r3, r0 => c'est l'adresse de la mémoire allouée (un pointeur)
```

```
| | | 0x000082f8 003084e5 str r3, [r4] => on store l'adresse dans r4. Donc r4 contient pointe sur cette adresse allouée
```

```
| | | 0x000082fc 1c301be5 ldr r3, [var_1ch] ; 0x1c ; 28 => on load la valeur contenue dans var1CH donc 0 donc r3 = 0
```

```
| | | 0x00008300 0321a0e1 lsl r2, r3, 2 => on décale r3 de 2 octets vers la gauche et on met le résultat dans r2. <<
```

```
| | | 0x00008304 20301be5 ldr r3, [var_20h] ; 0x20 ; 32 => on récupère la valeur dans var20 et on la met dans r3. Vu que var20h est un pointeur on récupère l'adresse de la mémoire allouée je pense.
```

```
| | | 0x00008308 023083e0 add r3, r3, r2 => on additionne l'adresse avec ce que contient r2 donc on se déplace dans la mémoire. et on met l'adresse dans r4
```

```
| | | 0x0000830c 003093e5 ldr r3, [r3] => on load la valeur contenue dans à l'adresse r3 dans r3. ou alors on récupère l'adresse de r3
```

```
| | | 0x00008310 0300a0e1 mov r0, r3 ; void *s => on met cette adresse dans r0. r0 pointe sur l'adresse de r3
```

```
| | | 0x00008314 0a10a0e3 mov r1, 0xa ; int c => on remplit le bloc de mémoire pointé par r3 (mtn r0) par 10.
```

```
| | | 0x00008318 2020a0e3 mov r2, 0x20 => on remplis de 32 octets la zone mémoire de la
```

valeur 10.

```
|  ||| 0x000831c 2f2700eb bl sym.memset ; void *memset(void *s, int c, size_t n)
|  ||| 0x0008320 1c301be5 ldr r3, [var_1ch] ; 0x1c ; 28 => on load la valeur contenue
dans var1ch donc 0 donc r3 = 0

|  ||| 0x0008324 013083e2 add r3, r3, 1 => on ajoute 1 à r3 donc ici r3 = 1
|  ||| 0x0008328 1c300be5 str r3, [var_1ch] ; 0x1c ; 28 => on store cette nouvelle valeur
dans var1ch

|  ||| ; CODE XREF from main @ 0x82d8(x)
|  ||| 0x000832c 1c301be5 ldr r3, [var_1ch] ; 0x1c ; 28 => on récupère cette nouvelle
valeur et on la remet dans r3

|  || 0x0008330 080053e3 cmp r3, 8 ; 8 => on compare la valeur stocké dans r3
avec 8

|  | 0x0008334 e8ffff1a bne 0x82dc => branch if not equal => si ce n'est pas égal à 8 on
recommence. Donc on est dans une boucle qui tourne 8 fois.

|  | 0x0008338 1c301be5 ldr r3, [var_1ch] ; 0x1c ; 28 => on récupère cette nouvelle
valeur et on la remet dans r3

|  | 0x000833c 0321a0e1 lsl r2, r3, 2 => on décale r3 de 4 car 2 puissance 2 = 4 octets vers la
gauche et on met le résultat dans r2. <<

|  | 0x0008340 20301be5 ldr r3, [var_20h] ; 0x20 ; 32 => on récupère la valeur dans
var20 et on la met dans r3. Vu que var20h est un pointeur on récupère l'adresse de la mémoire allouée
je pense.

|  | 0x0008344 022083e0 add r2, r3, r2 => on additionne l'adresse avec ce que contient r2
donc on se déplace dans la mémoire. et on met l'adresse dans r4

|  | 0x0008348 0030a0e3 mov r3, 0 => on met 0 dans r3

|  | 0x000834c 003082e5 str r3, [r2] => on store cette valeur dans r2

|  | 0x0008350 0030a0e3 mov r3, 0 ==> on remet 0 dans r3

|  | 0x0008354 1c300be5 str r3, [var_1ch] ; 0x1c ; 28 ==> on met var1ch à zéro. Donc
var1ch = 0

|  | 0x0008358 4130a0e3 mov r3, 0x41 ; 'A' ==> r3 = A

|  | 0x000835c 18300be5 str r3, [var_1ch] ; 0x18 ; 24 ==> on store A dans var1ch donc
var1ch = A

|  | 0x0008360 0d0000ea b 0x839c ==> on breach vers cette adresse

|  || ; CODE XREF from main @ 0x83a4(x)
|  | 0x0008364 20301be5 ldr r3, [var_20h] ; 0x20 ; 32 ==> on load la valeur de var20h
dans r3 donc Vu que var20h est un pointeur on récupère l'adresse de la mémoire allouée je pense.

|  ||| 0x0008368 0c3083e2 add r3, r3, 0xc ==> on fait r3+12 (c'est un offset) donc on
récupère l'adresse et on la met dans r3

|  ||| 0x000836c 002093e5 ldr r2, [r3] ==> on récupère la valeur qui se situe à l'offset r3+12
et on la met dans r2

|  ||| 0x0008370 1c301be5 ldr r3, [var_1ch] ; 0x1c ; 28 ==> on load dans r3 donc A

|  ||| 0x0008374 032082e0 add r2, r2, r3 ==> on additionne r2 = r2+r3

|  ||| 0x0008378 18301be5 ldr r3, [var_1ch] ; 0x18 ; 24 ==> on récupère la valeur
stocké dans var1ch donc A

|  ||| 0x000837c ff3003e2 and r3, r3, 0xff ==> on fait un AND avec 0xFF = -1 donc on garde je
crois que le dernier bits (ça fait quand meme A)

|  ||| 0x0008380 0030c2e5 strb r3, [r2] ==> on store le byte contenu dans r3 dans r2

|  ||| 0x0008384 18301be5 ldr r3, [var_1ch] ; 0x18 ; 24 ==> on load la valeur de var1ch
dans r3 on store A

|  ||| 0x0008388 013083e2 add r3, r3, 1 ==> on incrémente r3 de 1 donc A+1 = B

|  ||| 0x000838c 18300be5 str r3, [var_1ch] ; 0x18 ; 24 ==> on store cette valeur dans
var1ch on store B dans var1ch

|  ||| 0x0008390 1c301be5 ldr r3, [var_1ch] ; 0x1c ; 28 ==> on reload cette valeur dans
r3 on récupère B dans r3

|  ||| 0x0008394 013083e2 add r3, r3, 1 ==> on ajoute 1 encore dans r3 on ajoute 1 donc B
+ 1 = C

|  ||| 0x0008398 1c300be5 str r3, [var_1ch] ; 0x1c ; 28 ==> on stocke cette nouvelle
valeur dans var1ch = C

|  ||| ; CODE XREF from main @ 0x8360(x)
|  ||| 0x000839c 1c301be5 ldr r3, [var_1ch] ; 0x1c ; 28 ==> on load la valeur se
trouvant dans var1ch dans r3 donc C

|  || 0x00083a0 1f0053e3 cmp r3, 0x1f ; 31 ==> on compare 0x1f avec la valeur
contenue dans r3 donc 31 (15+16) on compare C avec 31 en ascii = 1

|  | 0x00083a4 eeffff1a bne 0x8364 => branch if not equal on retourne dans la boucle si la
comparaison n'est pas bonne

|  | 0x00083a8 20301be5 ldr r3, [var_20h] ; 0x20 ; 32 ==> si la comparaison est bonne
on récupère la valeur contenue dans var20h donc l'adresse de la mémoire allouée

|  | 0x00083ac 0c3083e2 add r3, r3, 0xc ==> on se déplace à l'offset 12 depuis l'adresse
mémoire pointée

|  | 0x00083b0 002093e5 ldr r2, [r3] ==> on load dans r2 la valeur contenu dans l'adresse
mémoire pointée

|  | 0x00083b4 1c301be5 ldr r3, [var_1ch] ; 0x1c ; 28 ==> on load dans r3 la valeur
contenue dans var1ch donc C à ce moment là

|  | 0x00083b8 032082e0 add r2, r2, r3 ==> on additionne la valeur contenue dans r3 avec r2
et on la stocke dans r2

|  | 0x00083bc 0030a0e3 mov r3, 0 ==> on met 0 dans r3

|  | 0x00083c0 0030c2e5 strb r3, [r2] ==> on store un byte de r3 dans r2

|  | 0x00083c4 0030a0e3 mov r3, 0 ==> on met 0 dans r3

|  | 0x00083c8 1c300be5 str r3, [var_1ch] ; 0x1c ; 28 ==> on store 0 dans var1ch donc
ici var1ch = 0

|  | 0x00083cc 130000ea b 0x8420 => jmp vers le good guy

|  || ; CODE XREF from main @ 0x843c(x)
|  | 0x00083d0 2c301be5 ldr r3, [var_2ch] ; 0x2c ; 44 ==> on load notre input (argv)
dans r3
```

```
|  ||| 0x00083d4 043083e2 add r3, r3, 4 ==> on ajoute 4 donc on se déplace à la 4 case ??
|  ||| 0x00083d8 002093e5 ldr r2, [r3] ==> on récupère la valeur stocké dans r3 et on la met
dans r2 donc r2 = r3
|  ||| 0x00083dc 1c301be5 ldr r3, [var_1ch] ; 0x1c ; 28 ==> on fait r3 = var1ch
|  ||| 0x00083e0 033082e0 add r3, r2, r3 ==> on fait r3 = r2 + r3
|  ||| 0x00083e4 0010d3e5 ldrb r1, [r3] ==> on load un byte de r3 dans r1 donc : r1 = r3[i]
|  ||| 0x00083e8 20301be5 ldr r3, [var_20h] ; 0x20 ; 32 ==> on fait r3 = var20h donc r3
récupère note input (donc r3 pointe sur le début de notre input
|  ||| 0x00083ec 0c3083e2 add r3, r3, 0xc ==> on se decale de 12 case dans r3 donc r3 pointe
sur r3[12]
|  ||| 0x00083f0 002093e5 ldr r2, [r3] ==> on load cette valeur dans r2
|  ||| 0x00083f4 1c301be5 ldr r3, [var_1ch] ; 0x1c ; 28 ==> on fait r3 = var1ch
|  ||| 0x00083f8 033082e0 add r3, r2, r3 ==> on fait r3 = r3+r2 hmmm
|  ||| 0x00083fc 0030d3e5 ldrb r3, [r3] ==> on récupère un byte de r3 et on le load dans r3
donc : r3 = r3[i]
|  ||| 0x0008400 030051e1 cmp r1, r3 ==> on compare r1 avec r3
|  |< 0x0008404 0200000a beq 0x8414 => branch if equal
|  ||| 0x0008408 0030e0e3 mvn r3, 0 => on fait un move NOT donc on bouge l'inverse du
résultat dans r3
|  ||| 0x000840c 30300be5 str r3, [var_30h] ; 0x30 ; 48 ==> on store cette valeur dans
var30ch
|  |< 0x0008410 0c0000ea b 0x8448 => jmp vers le bad guy
|  ||| ; CODE XREF from main @ 0x8404(x)
|  |> 0x0008414 1c301be5 ldr r3, [var_1ch] ; 0x1c ; 28 ==> on load la valeur se
trouvant dans var1ch dans r3
|  ||| 0x0008418 013083e2 add r3, r3, 1 ==> on incrémente de 1 la valeur contenue dans r3
|  ||| 0x000841c 1c300be5 str r3, [var_1ch] ; 0x1c ; 28 ==> on store cette valeur dans
var1ch
|  ||| ; CODE XREF from main @ 0x83cc(x)
|  ||| 0x0008420 2c301be5 ldr r3, [var_2ch] ; 0x2c ; 44 ==> on load notre input (argv)
dans r3
|  ||| 0x0008424 043083e2 add r3, r3, 4 ==> on se décale à la 4 eme case de r3
|  ||| 0x0008428 002093e5 ldr r2, [r3] ==> on récupère la valeur contenu dans cette case et
on la met dans r2
|  ||| 0x000842c 1c301be5 ldr r3, [var_1ch] ; 0x1c ; 28 ==> on load la valeur se trouvant
dans var1ch dans r3
|  ||| 0x0008430 033082e0 add r3, r2, r3 ==> on additionne la valeur contenue dans var2c
avec la valeur contenue dans r3. je pense que r3 contient un nombre et r2 contient un caractère ascii et
donc on additionne a chaque tour a certaine case
|  ||| 0x0008434 0030d3e5 ldrb r3, [r3] ==> on met cette nouvelle valeur (byte) dans r3
|  ||| 0x0008438 000053e3 cmp r3, 0 ==> on compare r3 avec 0
|  |< 0x000843c e3ffff1a bne 0x83d0 ==> branch if not equal
|  || 0x0008440 10309fe5 ldr r3, [0x0008458] ; [0x8458:4]=0x539 ==> good guys !
|  || 0x0008444 30300be5 str r3, [var_30h] ; 0x30 ; 48
|  || ; CODE XREFS from main @ 0x82bc(x), 0x8410(x)
|  |> 0x0008448 30301be5 ldr r3, [var_30h] ; 0x30 ; 48
|  || 0x000844c 0300a0e1 mov r0, r3
|  || 0x0008450 10d04be2 sub sp, fp, 0x10
|  || 0x0008454 10a89de8 ldm sp, {r4, fp, sp, pc}
```


